

ШПАРГАЛКИ ПО ROS2 ДЛЯ РАЗРАБОТЧИКОВ

ROS2 CLI КАК ИНСТРУМЕНТ ИССЛЕДОВАНИЯ СИСТЕМЫ ROS2

Разработано в РТУ МИРЭА ИИИ КПУ
Грачевым А.В.
Зениной А.А.

ОГЛАВЛЕНИЕ

Общая структура команд	3
1. Исследование узлов (Nodes)	3
2. Исследование топиков (Topics)	4
3. Исследование сервисов (Services)	6
4. Исследование экшенов (Actions)	8
5. Исследование параметров (Parameters)	11
6. Исследование пакетов (Packages)	15
7. Исследование интерфейсов (Interfaces)	16
8. Запуск и управление нодами	16
9. Диагностика системы	18
10. Просмотр графа системы (визуально)	20
Дополнительные полезные команды	20
Шпаргалка в таблице (быстрый доступ)	20
Обратная связь	22

Общая структура команд

Все команды исследования строятся по единому шаблону

```
ros2 <сущность> <действие> [аргументы] [опции]
```

Например: `ros2 node list`, `ros2 topic echo /chatter`,
`ros2 service call /add_two_ints ....`

Важно: перед использованием убедитесь, что система ROS2 запущена (хотя бы `ros2 daemon` работает, а ноды общаются через DDS). Установка

1. Исследование узлов (Nodes)

Узлы — это исполняемые процессы, которые общаются через топики, сервисы и экшены.

Команд	Описание	Пример
<code>ros2 node list</code>	Показать все активные узлы.	<code>ros2 node list</code>
<code>ros2 node info <node_name></code>	Показать подробную информацию об узле: подписчики, издатели, серверы сервисов, клиенты сервисов, серверы экшенов, клиенты экшенов.	<code>ros2 node info /turtlesim</code>

Пример вывода `ros2 node info /turtlesim`:

```
/turtlesim
Subscribers:
  /turtle1/cmd_vel: geometry_msgs/msg/Twist
  /parameter_events: rcl_interfaces/msg/ParameterEvent
Publishers:
  /turtle1/color_sensor: turtlesim/msg/Color
  /turtle1/pose: turtlesim/msg/Pose
  /rosout: rcl_interfaces/msg/Log
Service Servers:
  /turtle1/set_pen: turtlesim/srv/SetPen
  /turtle1/teleport_absolute: turtlesim/srv/TeleportAbsolute
  ...
Service Clients:
Action Servers:
  /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute
Action Clients:
```

Совет: имена узлов могут быть абсолютными (начинаться с `/`) или относительными. Всегда используйте полное имя, как выведено в `node list`.

2. Исследование топиков (Topics)

Топики — основные каналы обмена данными (публикация-подписка).

Команда	Описание	Пример
<code>ros2 topic list</code>	Список всех активных топиков.	<code>ros2 topic list</code>
<code>ros2 topic list -t</code>	Список топиков с указанием типа сообщения.	<code>ros2 topic list -t</code>
<code>ros2 topic info <topic_name></code>	Показать информацию о топике: тип и количество издателей/подписчиков.	<code>ros2 topic info /turtle1/pose</code>
<code>ros2 topic type <topic_name></code>	Показать только тип сообщения топика (удобно для подстановки в другие команды).	<code>ros2 topic type /turtle1/cmd_vel</code>
<code>ros2 topic echo <topic_name></code>	Вывести в консоль все сообщения, публикуемые в топик (нажмите Ctrl+C для выхода).	<code>ros2 topic echo /turtle1/pose</code>
<code>ros2 topic echo <topic_name> --csv</code>	Вывод в формате CSV (полезно для записи в файл).	<code>ros2 topic echo /turtle1/pose --csv</code>
<code>ros2 topic echo <topic_name> --field <field></code>	Вывести только конкретное поле сообщения.	<code>ros2 topic echo /turtle1/pose --field x</code>
<code>ros2 topic hz <topic_name></code>	Измерить частоту публикации сообщений (средняя, мин, макс за период).	<code>ros2 topic hz /turtle1/pose</code>
<code>ros2 topic hz <topic_name> --window <N></code>	Измерять частоту по скользящему окну из N сообщений.	<code>ros2 topic hz /turtle1/pose --window 10</code>
<code>ros2 topic bw <topic_name></code>	Измерить полезную пропускную способность (байт/сек).	<code>ros2 topic bw /turtle1/pose</code>
<code>ros2 topic delay <topic_name></code>	Измерить задержку между публикацией и получением (требует наличия временных меток в сообщении).	<code>ros2 topic delay /chatter</code>
<code>ros2 topic pub <topic_name> <type> <data></code>	Опубликовать одно сообщение в топик (полезно для тестирования).	<code>ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z: 0.0}}" --once</code>
<code>ros2 topic pub ... --rate <Hz></code>	Публиковать сообщения с заданной частотой.	<code>... --rate 10</code>
<code>ros2 topic find <type></code>	Найти все топики заданного типа.	<code>ros2 topic find geometry_msgs/msg/Twist</code>

Пример измерения частоты:

```
$ ros2 topic hz /turtle1/pose  
average rate: 62.434  
min: 0.016s max: 0.016s std dev: 0.000009s window: 62
```

Совет: используйте `--once` для однократной публикации, а `--rate` — для стресс-тестирования подписчиков.

3. Исследование сервисов (Services)

Сервисы реализуют модель запрос-ответ.

Команда	Описание	Пример
<code>ros2 service list</code>	Список всех активных сервисов.	<code>ros2 service list</code>
<code>ros2 service list -t</code>	Список сервисов с типами.	<code>ros2 service list -t</code>
<code>ros2 service type <service_name></code>	Показать тип сервиса.	<code>ros2 service type /spawn</code>
<code>ros2 service find <type></code>	Найти все сервисы заданного типа.	<code>ros2 service find turtlesim/srv/Spawn</code>
<code>ros2 service call <service_name> <type> <data></code>	Вызвать сервис с данными (в формате YAML).	<code>ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 2.0, theta: 0.2, name: 'turtle2'}"</code>

Пример вызова сервиса с ответом:

```
$ ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 5, b: 3}"
requester: making request: example_interfaces.srv.AddTwoInts_Request(a=5, b=3)

response:
example_interfaces.srv.AddTwoInts_Response(sum=8)
```

Важно: формат данных — YAML в одну строку. Для сложных структур можно использовать многострочный ввод (оберните в кавычки).

4. Исследование экшенов (Actions)

Экшены — это долговременные задачи с обратной связью и возможностью прервать.

Команда	Описание	Пример
<code>ros2 action list</code>	Список всех активных экшенов.	<code>ros2 action list</code>
<code>ros2 action list -t</code>	Список экшенов с типами.	<code>ros2 action list -t</code>
<code>ros2 action info <action_name></code>	Показать информацию о клиентах и серверах экшена.	<code>ros2 action info /turtle1/rotate_absolute</code>
<code>ros2 action type <action_name></code>	Показать тип экшена.	<code>ros2 action type /turtle1/rotate_absolute</code>
<code>ros2 action send_goal <action_name> <type> <data></code>	Отправить цель экшену (выводит результат).	<code>ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.57}"</code>
<code>ros2 action send_goal ... --feedback</code>	Отображать обратную связь во время выполнения.	<code>... --feedback</code>

Пример с обратной связью:

```
$ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.57}" --feedback
Waiting for an action server to become available...
Sending goal:
  theta: 1.57

Feedback:
  remaining: 1.57
  ...
Result:
  delta: 0.0
```

Goal finished with status SUCCEEDED

5. Исследование параметров (Parameters)

Параметры позволяют настраивать поведение узлов во время выполнения.

Команда	Описание	Пример
<code>ros2 param list</code>	Список всех параметров всех узлов.	<code>ros2 param list</code>
<code>ros2 param list <node_name></code>	Список параметров конкретного узла.	<code>ros2 param list /turtlesim</code>
<code>ros2 param get <node_name> <param_name></code>	Получить значение параметра.	<code>ros2 param get /turtlesim background_r</code>
<code>ros2 param set <node_name> <param_name> <value></code>	Установить значение параметра (изменяется только в рантайме, если узел поддерживает).	<code>ros2 param set /turtlesim background_r 150</code>
<code>ros2 param describe <node_name> <param_name></code>	Показать описание параметра (тип, ограничения, описание), если узел его предоставляет.	<code>ros2 param describe /turtlesim background_r</code>
<code>ros2 param dump <node_name></code>	Вывести все параметры узла в формате YAML.	<code>ros2 param dump /turtlesim</code>
<code>ros2 param dump <node_name> > params.yaml</code>	Сохранить параметры в файл.	<code>ros2 param dump /turtlesim > turtlesim_params.yaml</code>
<code>ros2 param load <node_name> <file></code>	Загрузить параметры из файла YAML.	<code>ros2 param load /turtlesim turtlesim_params.yaml</code>
<code>ros2 param delete <node_name> <param_name></code>	Удалить параметр (если узел поддерживает динамическое создание/удаление).	<code>ros2 param delete /my_node foo</code>

Формат файла параметров:

```
/turtlesim:
  ros__parameters:
    background_b: 255
    background_g: 86
    background_r: 150
    qos_overrides:
      /parameter_events:
```

```
publisher:  
  depth: 1000  
  ...
```

Примечание: многие параметры могут быть изменены только при запуске через `--ros-args -p`. Команда `set` меняет их только в runtime, если узел реализует callback параметров.

6. Исследование пакетов (Packages)

Интерфейсы — это определения типов сообщений, сервисов и экшенов.

Команд	Описание	Пример
<code>ros2 pkg list</code>	Список всех установленных пакетов.	<code>ros2 pkg list</code>
<code>ros2 pkg executables <package_name></code>	Список всех исполняемых файлов (нод) в пакете.	<code>ros2 pkg executables turtlesim</code>
<code>ros2 pkg prefix <package_name></code>	Показать путь установки пакета.	<code>ros2 pkg prefix turtlesim</code>
<code>ros2 pkg xml <package_name></code>	Вывести содержимое package.xml пакета.	<code>ros2 pkg xml turtlesim</code>

`ros2 pkg executables` очень полезен, чтобы узнать, какие именно ноды предоставляет пакет.

7. Исследование интерфейсов (Interfaces)

Пакеты — это организационные единицы кода в ROS2.

Команда	Описание	Пример
<code>ros2 interface list</code>	Список всех доступных типов интерфейсов (можно фильтровать по пакету).	<code>ros2 interface list</code>
<code>ros2 interface package <package_name></code>	Показать все интерфейсы в указанном пакете.	<code>ros2 interface package turtlesim</code>
<code>ros2 interface packages</code>	Список пакетов, содержащих интерфейсы.	<code>ros2 interface packages</code>
<code>ros2 interface show <type></code>	Показать структуру сообщения/сервиса/экшена.	<code>ros2 interface show geometry_msgs/msg/Twist</code>
<code>ros2 interface proto <type></code>	Показать пример заполнения (прототип) для использования в <code>pub</code> или <code>call</code> .	<code>ros2 interface proto geometry_msgs/msg/Twist</code>

Пример `show`:

```
$ ros2 interface show geometry_msgs/msg/Twist
# This expresses velocity in free space broken into its linear
and angular parts.
Vector3 linear
Vector3 angular
```

Пример `proto`:

```
$ ros2 interface proto geometry_msgs/msg/Twist
"linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0"
```

8. Запуск и управление нодам

Команда	Описание	Пример
<code>ros2 run <package_name> <executable_name></code>	Запустить ноду из пакета.	<code>ros2 run turtlesim turtlesim_node</code>
<code>ros2 run <package_name> <executable_name> --ros-args -p param:=value</code>	Передать параметры при запуске.	<code>ros2 run turtlesim turtlesim_node --ros-args -p background_r:=200</code>
<code>ros2 run <package_name> <executable_name> --ros-args -r old:=new</code>	Переименовать топики/сервисы при запуске.	<code>ros2 run turtlesim turtlesim_node --ros-args -r /turtle1/pose:=/turtle2/pose</code>
<code>ros2 lifecycle list</code>	Показать узлы с поддержкой управления жизненным циклом.	<code>ros2 lifecycle list</code>
<code>ros2 lifecycle get <node_name></code>	Получить текущее состояние узла жизненного цикла.	<code>ros2 lifecycle get /lc_node</code>
<code>ros2 lifecycle set <node_name> <transition></code>	Перевести узел в другое состояние (configure, activate, deactivate, cleanup, shutdown).	<code>ros2 lifecycle set /lc_node configure</code>

9. Диагностика системы

Команда	Описание	Пример
<code>ros2 doctor</code>	Запустить все диагностические проверки (состояние сети, демона, конфигурации).	<code>ros2 doctor</code>
<code>ros2 doctor --report</code>	Вывести полный отчёт (рекомендуется при проблемах).	<code>ros2 doctor --report</code>
<code>ros2 wtf</code>	То же, что и <code>ros2 doctor --report</code> (псевдоним).	<code>ros2 wtf</code>
<code>ros2 multicast receive</code>	Проверить, получает ли система мультикаст-сообщения (для отладки DDS discovery).	<code>ros2 multicast receive</code>
<code>ros2 multicast send</code>	Отправить тестовое мультикаст-сообщение.	<code>ros2 multicast send</code>
<code>ros2 daemon status</code>	Проверить, запущен ли демон <code>ros2</code> .	<code>ros2 daemon status</code>
<code>ros2 daemon stop</code>	Остановить демон (иногда полезно при зависаниях).	<code>ros2 daemon stop</code>
<code>ros2 daemon start</code>	Запустить демон заново.	<code>ros2 daemon start</code>
<code>ros2 security list</code>	Показать список включённых политик безопасности (если используется <code>SR0S2</code>).	<code>ros2 security list</code>

`ros2 doctor` — твой первый помощник при странных ошибках: он проверит версии, переменные окружения, доступность демона и даже конфликты портов.

10. Просмотр графа системы (визуально)

Хотя это не CLI в чистом виде, но `rqt_graph` — неотъемлемый инструмент для исследования. Запускается из командной строки:

```
rqt_graph
```

Или

```
ros2 run rqt_graph rqt_graph
```

Он показывает ноды и топики в виде графа, что очень наглядно.

Дополнительные полезные команды

Команда	Описание
<code>ros2 topic echo /rosout</code>	Посмотреть логи всех нод в реальном времени.
<code>ros2 node list xargs -I {} ros2 node info {}</code>	Получить информацию обо всех нодах сразу (bash).
<code>ros2 param list grep background</code>	Поиск параметров по имени.
<code>ros2 interface show \$(ros2 topic type /chatter)</code>	Узнать тип сообщения топика и сразу посмотреть его структуру.
<code>ros2 topic echo /some_topic grep -m 1 "field"</code>	Вывести только первое вхождение поля.
<code>ros2 bag record -a -o debug_bag &</code>	Записать всё в фоне, пока тестируешь систему.

Шпаргалка в таблице (быстрый доступ)

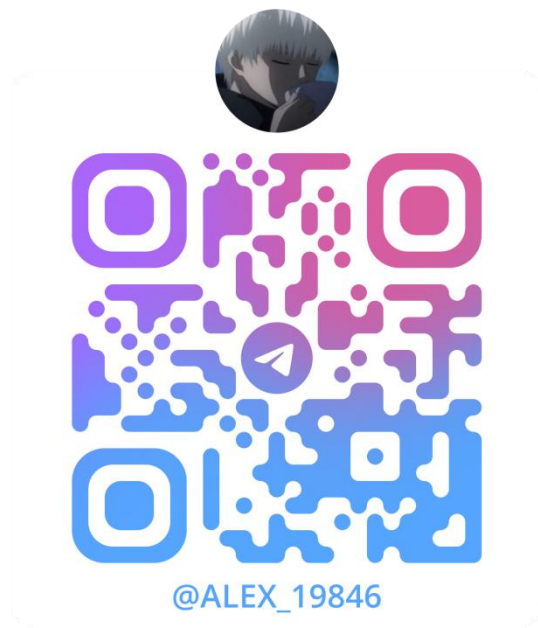
Сущность	Команда	Действие
Узлы	<code>ros2 node list</code>	Список узлов
	<code>ros2 node info <node></code>	Детали узла
Топики	<code>ros2 topic list [-t]</code>	Список топиков (с типами)
	<code>ros2 topic echo <topic></code>	Вывод сообщений
	<code>ros2 topic hz <topic></code>	Частота сообщений
	<code>ros2 topic pub <topic> <type> <data></code>	Публикация тестового сообщения
	<code>ros2 service list [-t]</code>	Список сервисов
Сервисы	<code>ros2 service call <service> <type> <data></code>	Вызов сервиса
	<code>ros2 action list [-t]</code>	Список экшенов
Экшены	<code>ros2 action send_goal <action> <type> <data></code>	Отправка цели
	<code>ros2 param list</code>	Список параметров
Параметры	<code>ros2 param get <node></code>	Получить значение

	<param>	
	ros2 param set <node> <param> <value>	Установить значение
	ros2 param dump <node>	Выгрузить все параметры
Интерфейсы	ros2 interface show <type>	Показать структуру
	ros2 interface proto <type>	Пример заполнения
Пакеты	ros2 pkg executables <pkg>	Исполняемые файлы пакета
Диагностика	ros2 doctor	Проверка здоровья системы
	ros2 multicast receive	Проверка мультикаста

Обратная связь

Если Вы нашли ошибку, неточность, у Вас есть предложения по улучшению или вопросы, напишите в Telegram Александру (https://t.me/Alex_19846) или Алисе (https://t.me/Kika_01). QR-коды со ссылками на их аккаунты представлены ниже.

Александр



Алиса

